When Can Static Verification Help Agent Workflows? A Base-Rate and Model-Fidelity Study

Anonymous submission

Abstract

Agent frameworks increasingly encode tool-using behavior as explicit workflow graphs whose topology can be analyzed before deployment—yet how often such graphs actually contain the structural defects static verification targets has not been measured. We build AGENTPROOF, which automatically extracts a unified graph model with per-element extraction provenance from four agent frameworks (LangGraph, CrewAI, AutoGen, Google ADK) with no per-workflow manual modeling, applies six structural checks with witness traces, and compiles a temporal policy DSL with LTL_f (finite-trace) semantics—differentially tested against an independent reference oracle—to DFAs evaluated at runtime and statically over finite maximal executions, via a conservative graph $\times$ DFA product that explores every declared tool binding, covers bad prefixes and end-of-trace obligations, and returns an explicit *inconclusive* verdict for behavior it cannot certify. We use it as a measurement instrument on 922 workflows from 252 public GitHub repositories, with a 119-workflow sample validated against independently reconstructed reference graphs (LLM-generated, adversarially cross-checked, not human annotations) and adversarial triage—reporting flag *precision* and defect *prevalence* as separate estimands, since a lossy extractor both manufactures false flags and hides true defects. Flags are almost never defects: 2 of 186 are genuine (1.1%); every structural flag is an extraction artifact. But re-running the checks on the reference graphs reveals defects the extractor missed, doubling measured prevalence to 4 of 119 workflows (3.4%, 95% CI $[1.3, 8.3]$%)—concentrated in application-like workflows binding side-effecting tools (7.7%) rather than the tutorials and demos that dominate public GitHub (1.3%). We diagnose extraction fidelity (edge recall 0.65 on real code, 0.38 on implicit-topology ADK), not the check algorithms, as the binding constraint, and give the static machinery a defect-independent job under an explicit trace-containment premise: proving which runtime monitors can never fire, so they can be soundly pruned (88% on a curated corpus, all alphabet-level today).

1 Introduction

Large language models are increasingly deployed as autonomous agents that call external tools and APIs (Schick et al. 2023; Yao et al. 2023). Several orchestration frameworks structure this behavior as explicit *workflow graphs*: LangGraph (LangChain 2025), CrewAI (CrewAI 2025), AutoGen (Wu et al. 2024), and Google ADK (Google 2025) each represent computation steps as nodes and allowable transitions as edges. Yet safety enforcement in these systems remains almost entirely a *runtime* concern: tools such as NeMo Guardrails (NVIDIA 2023) and LlamaGuard (Inan et al. 2023) filter individual calls at execution time, detecting a violation only when the offending path is exercised.

Some defects, however, are topological. Consider an email-triage agent whose developer adds a `draft_response` node but forgets to connect it to `send`: normal-priority emails silently halt at a dead end. A content-level runtime guardrail cannot catch this unless that path runs, but a static analysis of the graph flags the dead end and emits a witness trace. In principle classical model checkers (Clarke, Grumberg, and Peled 1999; Holzmann 1997; Cimatti et al. 2002) verify such properties, but they require hand-translating the system into Promela or SMV—prohibitive for developers iterating on workflows. For conventional static analyzers, decades of deployment experience show that what decides adoption is not detection power but the rate of genuine, actionable findings (Bessey et al. 2010; Johnson et al. 2013; Sadowski et al. 2018). No analogous measurement exists for agent workflows.

We build AGENTPROOF to make that measurement. It extracts a unified graph model from four frameworks' APIs with no per-workflow manual modeling, runs six structural checks with witness traces, and compiles a temporal policy DSL with LTL_f (finite-trace) semantics (De Giacomo and Vardi 2013) to DFAs checked statically via a graph $\times$ DFA product and enforced at runtime. We then use it as a measurement instrument on 922 workflow graphs mined from 252 public GitHub repositories, with a stratified 119-workflow sample validated against independently reconstructed reference graphs and adversarial triage. Four questions organize the study: how faithfully workflow behavior can be recovered from source (RQ1), what fraction of static findings is genuine (RQ2), what defect prevalence remains after auditing false negatives (RQ3), and when static analysis can soundly reduce runtime enforcement (RQ4).

Because a lossy extractor both *manufactures* false flags and *hides* true defects, we report two estimands separately:

the *precision* of the instrument's flags (what fraction of raised flags are genuine) and the *prevalence* of defects per workflow, estimated by re-running every check on the reconstructed reference graphs so that defects the extractor missed are counted. The distinction matters: flag precision is 1.1% (2 of 186), but prevalence is roughly twice what the extractor sees—4 of 119 workflows (3.4%)—because two genuine defects, including an SQL-executing tool reachable with no human gate, were invisible to the lossy extraction. Both are small; neither is the ≈ 0 a naïve reading of flag counts would suggest, and the defects concentrate exactly where they matter: in application-like workflows binding side-effecting tools (3 of 39, versus 1 of 80 among tutorials, demos, and tests).

Contributions.

- **A base-rate study of topology defects in public agent workflows**, with the estimands kept honest: flag precision *and* workflow-level prevalence including false negatives, measured against independent graph reconstructions, with repository-clustered uncertainty and a source-level census of the corpus (one third application-like, two thirds tutorials/demos/tests; Section 3).
- **AGENTPROOF**, the measurement instrument: automatic extraction of a unified graph model with per-element provenance from four heterogeneous frameworks, six structural checks with witness traces, and a seven-form policy DSL with precise LTL$_f$ semantics—differentially tested against an independent reference oracle—whose conservative static product covers bad prefixes and end-of-trace obligations over finite maximal executions, explores every declared tool binding, and returns *inconclusive* rather than a proof when a cycle could postpone an obligation forever (Section 2).
- **A quantified diagnosis**: extraction fidelity, not the check algorithms, is the binding constraint. Edge recall is 0.65 on real code (0.38 on ADK) versus near-perfect on stubs; every structural flag on extracted graphs is an extraction artifact, and structural flags arise only for the one extractor that does not impose connected topologies—so extractor-level structural conclusions are meaningful for LangGraph only (Section 3).
- **A defect-independent job** for the static machinery, stated with its premise: given a graph that over-approximates runtime traces, the product proves which policy monitors can never fire, so they can be soundly pruned. On the curated corpus 237 of 270 monitor instances (88%) are provably inert and 14 are declined as inconclusive—though on today's small workflows every proven case is alphabet-level, so the product's value there is soundness on branching topologies, not extra pruning (Section 4).

2 The Agentproof System

Unified graph model. Agentproof represents a workflow as a directed graph $G = (V, E, v_0, V_f)$ with a distinguished entry v_0 and exit set V_f. Each node carries a kind $\kappa_V(v) \in \{\text{ENTRY}, \text{EXIT}, \text{TOOL}, \text{LLM}, \text{ROUTER}, \text{HUMAN}, \text{SUBGRAPH}\}$ and, for tool nodes, a set of bound tool names; each edge is DIRECT, CONDITIONAL, PARALLEL, or LOOP. Every node and edge

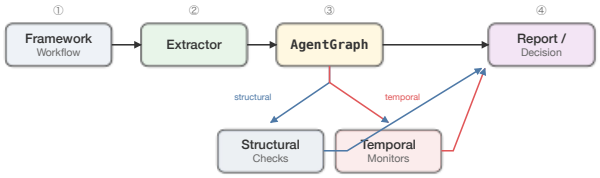

Figure 1: Agentproof extracts a unified graph model from four frameworks, runs structural checks and DFA-compiled temporal policies statically (and at runtime), and emits witness traces.

additionally carries extraction *provenance*—an origin (runtime object, explicit AST, inferred, synthesized, unknown), a confidence (exact, may, heuristic), and a source span—so that structure read from a framework declaration is distinguishable from structure the extractor guessed, and downstream witness traces inherit the weakest confidence they traverse. Per-framework extractors bridge four heterogeneous representations—each with different entry/exit conventions, edge semantics, and hierarchy models—into this single typed model. The extractors themselves encode framework semantics and are hand-written once per framework; what is eliminated is the per-workflow manual translation that SPIN or NuSMV require. Extraction has two paths: a runtime extractor that reads the compiled graph object, and a static AST fallback (used for large-scale mining, where installing every repository's dependencies is infeasible).

Structural checks. Six checks run over the topology, each in $O(|V| + |E|)$ and each emitting a *witness trace* on failure. *Exit reachability* verifies $V_f \subseteq \text{Reach}(v_0)$; *reverse reachability* flags reachable nodes with no path to any exit—executions entering such a node can never complete (passing this check does not guarantee termination, which depends on runtime routing); *dead-end detection* flags non-exit nodes with no successor; *router shape* is a framework-convention check requiring router outgoing edges to be conditional; *human presence* flags workflows with no HUMAN node (a deliberately blunt policy check we evaluate against a risk-aware variant); and *tool declaration* flags tool nodes with an empty tool set. A seventh, path-sensitive check, *human-gate coverage*, is the risk-aware variant: it flags a workflow only if a *sensitive* tool is reachable from v_0 without passing a HUMAN node (Section 3). Witness traces follow standard model-checking practice (Clarke, Grumberg, and Peled 1999) and pinpoint the offending path.

Temporal policies with LTL$_f$ semantics. Beyond topology, Agentproof supports a policy DSL over finite traces (LTL$_f$ (De Giacomo and Vardi 2013)): a workflow execution is a finite event sequence, and each of the seven DSL forms denotes an LTL$_f$ formula—forbidden ($\mathbf{G}\,\neg a$), response ($\mathbf{G}(a \rightarrow \mathbf{F}\,b)$), strong until ($a\,\mathbf{U}\,b$, including nested until), bounded response ($\mathbf{G}(a \rightarrow \mathbf{F}_{\leq k}\,b)$), response chain ($\mathbf{G}(a \rightarrow \mathbf{F}(b \wedge \mathbf{F}c))$), conjunction, and disjunction. Formulas are parsed into a typed recursive AST and compiled

compositionally to a DFA by LTL$_f$ formula progression (2–3 states for base patterns, $k{+}2$ for bounded response), with two disjoint violation mechanisms: *violation states* are exactly the doomed states—those from which no accepting state is reachable—so a bad prefix is flagged at the earliest event that makes every extension violating; and a trace ending in a non-accepting state—a pending **F**-obligation—violates the property at termination. The compiler's verdicts are differentially tested against an independent, directly recursive reference evaluator, exhaustively over all traces up to length six for two-atom formulas and property-based beyond. Runtime monitors evaluate events in $O(1)$ each and apply the end-of-trace acceptance check when the workflow terminates; obligations still open when a run is aborted are reported as inconclusive rather than violated, following three-valued runtime verification (Bauer, Leucker, and Schallhart 2011).

Static temporal verification. Policies are checked statically via a graph $\times$ DFA product: BFS from (v_0, q_0) advances the DFA at each node and follows graph edges. Executions are modeled as finite *maximal* paths, judged by LTL$_f$: paths ending at an exit node or at a dead end. Because static extraction cannot prove which of a tool node's declared tools will be invoked, or whether any will be, the product is *conservative*: at each tool node it closes the DFA state over every finite invocation sequence drawn from the declared tools, including the empty one—a false alarm is acceptable, a false proof is not. The checker returns one of three verdicts: *may-violate*, when some resolution reaches a DFA violation state (bad prefix) or a termination point with a pending obligation; *safe*, when no resolution does; and *inconclusive*, when the only finding is a reachable product cycle on which an obligation could be postponed forever, or no termination point is reachable—cases a finite-trace semantics cannot decide, which we deliberately do not claim. Witness paths carry the provenance-derived confidence of the edges and bindings they traverse. Explored states are bounded by $|V| \cdot |Q|$, so verification is linear in graph size for fixed-size DFAs.

Soundness and its premise. If the analyzed graph's paths *over-approximate* the workflow's runtime traces (trace containment), then a *safe* verdict implies the property holds on every finite maximal runtime execution—the guarantee we exploit for monitor pruning in Section 4. Trace containment holds by construction for our curated corpus (graphs authored with the code) and for runtime-extracted graphs absent dynamic graph modification. It does *not* hold for the lossy AST fallback used in mining, whose measured edge recall of 0.65 means extracted graphs *under*-approximate real behavior (Section 3); we therefore never claim static guarantees on mined graphs, only measurements—and the provenance fields make the distinction machine-checkable in the tool's output.

3 Base-Rate Study on Public GitHub Workflows

To estimate defect base rates on independently authored workflows—and to measure extractor fidelity on code we did not write—we mined public GitHub repositories across all four frameworks. We state the population precisely: *public GitHub files matching framework-specific code searches*, not production deployments; we census what that population contains below.

Mining and corpus census. Using GitHub code search (via the `gh` CLI, July 2026, with per-framework query sets and per-repository caps) we streamed each candidate repository through clone, extraction, and cleanup. After removing one repository of our own test fixtures (8 files; a contamination we detected during revision and exclude from every number), the corpus is 922 workflow graphs from 252 distinct repositories (LangGraph 400, AutoGen 359, CrewAI 113, ADK 50), with provenance recorded as repository, commit SHA, and file path. Mined workflows are small—median 2 non-sentinel nodes (mean 3.9)—and predominantly static linear pipelines. The corpus is *not* mostly applications: 32% of files carry tutorial/test/demo-style names, 48 topology groups are topologically identical duplicates across repositories (170 redundant workflows, retained but reported), and a source-level classification of the validated sample (below) found 44% tutorial or course material, 17% toy demos, 7% test files, and 33% application-like code. We therefore report prevalence both pooled and stratified by this classification.

Validation sample and reference graphs. On a stratified 119-workflow sample from 87 repositories (40 LangGraph, 20 CrewAI, 35 AutoGen, 24 ADK) we ran two independent LLM-agent passes (Claude Opus 4.8 orchestrated by a multi-agent harness; all outputs released). First, an agent reconstructed a reference graph from source alone (blind to the extractor's output, following its node-id conventions), with a self-reported confidence (77 high, 33 medium, 9 low); we score node/edge precision–recall and node-kind accuracy against these reconstructions. Second, every flagged defect was triaged from source (REAL/EXTRACTION-ARTIFACT/INTENTIONAL) and an adversarial verifier attempted to overturn each label (4.6% overturned; unresolved disagreements recorded as ARGUABLE). We acknowledge the limitation squarely: the reference graphs are LLM reconstructions, not human annotations, and both passes share a model family. Three mitigations bound the risk: reconstruction confidence is reported and results are stable on the high-confidence subset (edge recall 0.67 vs. 0.65 overall); every triage label carries a source-line-cited justification that survived an independent adversarial pass; and the label pipeline was re-audited during revision (one label overturned of 16 re-examined). Human annotation of a subsample is the clear next step. We report Wilson 95% CIs for proportions and repository-clustered bootstrap CIs (10,000 resamples) for means, since workflows from one repository are not independent.

Extraction fidelity is the bottleneck. Node detection stays strong (overall $P{=}0.91$, $R{=}0.85$), but edge recall falls to 0.649 and node-kind accuracy to 0.828, ranging from 0.38 edge recall on ADK (cluster CI $[0.14, 0.62]$) to 0.77 on AutoGen (Table 1). The frameworks that encode topology *implicitly*—ADK's nested `SequentialAgent`/

Table 1: AST-extractor fidelity on real workflows (n=119), with repository-clustered bootstrap 95% CIs on the overall means. Perfect on stubs; on real code fidelity is lower and strongly framework-dependent.

Framework	n	Edge P	Edge R	Kind acc
LangGraph	40	0.956	0.682	0.647
CrewAI	20	0.700	0.692	0.929
AutoGen	35	0.957	0.770	0.900
ADK	24	0.420	0.382	0.940
Overall	**119**	**0.805** [0.72, 0.89]	**0.649** [0.56, 0.73]	**0.828** [0.78, 0.87]

`ParallelAgent` composition, AutoGen's participant lists with dynamic speaker selection—are hardest: the extractor must guess an edge set the source never states, and for ADK it is wrong more often than right. On LangGraph, whose edges are explicit, the dominant error is instead node-kind (`tool`→`llm`: nodes invoking tools in their body with no declared binding). This 0.65 is a property of the static AST fallback used at scale, not of the runtime extractors, which read the compiled framework object: on trusted runnable examples the fallback recovers 0.71 of the runtime extractor's LangGraph edges (matching its 0.68 on mined code), and for ADK the runtime graph is far richer (16 vs. $\sim$5 edges). Realizing that ceiling at scale would require sandboxed execution of untrusted repository code, which we do not attempt. A post-hoc audit of the extractor itself illustrates the diagnosis *causally*: it silently dropped conditional edges declared through LangGraph's common three-argument `add_conditional_edges` form, so 209 of the 930 mined graphs contained a router with no outgoing edge while only 10 contained any conditional edge. Fixing that one bug and re-extracting at the identical pinned commits—the check algorithms untouched—raises LangGraph edge recall from 0.68 to 0.82 and removes roughly a quarter of the structural flags (from 314 to 239 flagged workflows, with unreachable-exit flags falling from 86 to 32), directly confirming extraction, not the checks, as the bottleneck. The fidelity and flag numbers in Table 1 and Table 2 describe the instrument as run during mining; the corrected re-extraction is released alongside it.

Estimand 1: flag precision. Of the 186 flags raised on extracted graphs, 2 (1.1%, Wilson 95% CI $[0.3, 3.8]\%$) are genuine—both missing human gates on sensitive workflows—while 42% are extraction artifacts and 50% intentional design (Table 2). *Every one of the 72 structural flags is an extraction artifact or arguable* (0 genuine; CI $[0, 5.1]\%$). Structural flags are also entirely a LangGraph phenomenon: they fire on 78% of LangGraph workflows but 0% of AutoGen, CrewAI, and ADK—and this is partly *by construction*, since those three extractors emit connected topologies and cannot produce a dead end or unreachable exit. Extractor-level structural results are therefore evidence about LangGraph only; for the other frameworks the structural checks are vacuous on extracted graphs, which is exactly why the next estimand is measured on reconstructions instead.

Table 2: Adversarial triage of the 186 flags the AST-extracted study raised. Every structural flag is an extraction artifact; the blunt human-presence flag is predominantly intentional. These rows measure flag *precision*, not defect prevalence.

Check	Real	Artifact	Intent.	Arg.
exit_reachability	0	14	0	0
reverse_reachability	0	25	0	0
dead_ends	0	24	0	1
router_shape	0	3	0	0
tool_declarations	0	5	0	0
human_presence	2	8	93	10
human_gate_coverage	0	0	0	1
Total	**2**	**79**	**93**	**12**

Table 3: Defect prevalence (workflows with $\geq$1 genuine defect), Wilson 95% CIs. The extractor-visible row is what flag triage alone supports; the reference-graph row adds defects found only on reconstructed graphs.

Estimate	Rate	95% CI
Extractor-visible, per workflow	2/119 (1.7%)	$[0.5, 5.9]\%$
+ reference graphs, per workflow	4/119 (3.4%)	$[1.3, 8.3]\%$
per repository	4/87 (4.6%)	$[1.8, 11.2]\%$
application-like only	3/39 (7.7%)	$[2.7, 20.3]\%$
tutorials/demos/tests	1/80 (1.3%)	$[0.2, 6.8]\%$

Estimand 2: prevalence, including what the extractor misses. Flag triage cannot see false negatives: a defect the lossy extractor never surfaces is simply absent from Table 2. We therefore re-ran every check on the 119 reconstructed reference graphs—which can contain dead ends and unreachable exits for all four frameworks—and adversarially triaged all 16 resulting flags (labels: genuine, intentional, or reconstruction error). Ten are intentional (e.g. deliberate wait-for-input halt nodes in multi-turn chat loops), four are reconstruction errors, and **two are genuine defects the extracted study missed**: a router mis-wiring in an ADK SQL agent, and a chatbot whose SQL-executing tool is reachable with no human gate. Combining confirmed defects from both passes (the two originals were re-confirmed by a second adversarial pass) gives the prevalence estimates of Table 3: 4 of 119 workflows (3.4%), 4 of 87 repositories (4.6%)—about *twice* what flag triage alone shows, quantifying the false-negative rate of the lossy instrument. All four defective workflows bind side-effecting tools, and three of four are application-like: prevalence is 7.7% among application-like workflows versus 1.3% among tutorials, demos, and tests. Topology defects are rare in this population, but not artifact-of-the-instrument rare, and they concentrate where the stakes are. Prespecified sensitivity analyses (supplement) leave every headline estimate materially unchanged: removing the 170 redundant duplicates shifts no rate by more than 3 points, repository-equal weighting agrees with the pooled estimates, and counting every unresolved (ARGUABLE) label pessimistically as genuine bounds prevalence at 12.6% of workflows.

Risk-aware gating. The blunt human-presence check fires on 91% of mined workflows with 2 of 113 firings genuine—uninformative as a detector, which is why we evaluate the risk-aware *human-gate coverage* check: flag only when a sensitive tool (destructive/write/exec/communication/financial, by a keyword lexicon over bound tool names) is reachable without a HUMAN gate. On the 922 mined graphs it fires on zero—but this is a statement about extracted *declarations*, not about the code: no mined graph declares a sensitive binding, while source-level classification finds side-effecting tools in 32 of 119 sampled workflows, invisible to the extractor through the `tool→llm` kind error. On the reconstructed graphs, where bindings are recovered from source, the same check fires three times and one firing is the genuine SQL-gate defect above. The check's logic is sound; its recall is bounded by extraction fidelity, which is the paper's central diagnosis.

4 Controlled Evaluation and Monitor Selection

Controlled detection test. A curated corpus of 18 workflows spanning all four frameworks, seeded with known anti-patterns, confirms the checks fire when defects are genuinely present: Agentproof detects every injected structural defect and, with a risk-aware sensitive-tool set, flags 8 workflows whose sensitive path (e.g. `account_create`, `publish`, `auto_remediate`) is reachable without a human gate, versus 10 for the blunt human-presence check—declining to flag two workflows that lack a human node but perform no sensitive action. This corpus is a controlled test that the algorithms work, not evidence of prevalence. Verification is sub-second even for synthetic graphs of 5,000 nodes (far larger than any observed workflow); details, per-framework extractor accuracy on stubs, and a Promela/SMV modeling-effort comparison are in the supplementary material.

Monitor selection via static pruning. The graph $\times$ DFA product gives the static machinery a job that does not depend on defects existing: deciding which runtime monitors to deploy. The soundness premise of Section 2 is required and we restrict the claim accordingly—pruning applies to graphs whose paths over-approximate runtime traces (here, the curated corpus; in deployment, runtime-extracted graphs), *never* to lossy AST-extracted graphs, whose missing edges could make a live monitor look inert. Under that premise, a policy whose product is *safe*—no resolution of the workflow's declared tool bindings reaches a bad prefix or a termination point with a pending obligation—can never fire at runtime on that workflow, so its monitor need not be deployed. The product natively explores every declared tool and the no-invocation case at each tool node, so unresolved tool choice over-approximates rather than fixing one arbitrary order; and it prunes only on *safe*: a workflow whose cycles could postpone an obligation forever yields *inconclusive*, and that monitor is kept.

Across the curated corpus and 15 policies, 237 of 270 monitor instances (87.8%) are provably inert—a mean of 13.2 of 15 monitors skipped per workflow—while 19 may fire and 14 (all on one seeded livelock workflow) are declined as inconclusive, leaving runtime enforcement to evaluate the remaining 12.2%.

We report two honest limits. First, on today's small, mostly linear workflows *every* proven-inert case is alphabet-level (the policy's atoms never appear in the workflow); under the corrected LTL_f semantics the product proves no additional case on this corpus—the single case the reachability product "won" under our earlier, incorrect semantics was in fact a live termination-obligation monitor. The product's value over a plain symbol-set check is therefore soundness, not extra pruning: on branching and looping topologies a symbol-set check cannot certify inertness while the product can, and the product is what licenses the guarantee at all. Second, the fixed policy vocabulary does not match real code—none of the 922 mined workflows invokes any of the 15 policies' tools—so measuring pruning on mined workflows requires instantiating policy templates against each workflow's own tools, which we leave to future work. Static monitor selection is thus a sound, cheap optimization whose measured leverage today is alphabet-level, and it is the concrete bridge between pre-deployment analysis and leaner runtime enforcement.

5 Related Work

Runtime agent safety. NeMo Guardrails (NVIDIA 2023), Guardrails AI (Guardrails AI 2023), LlamaGuard (Inan et al. 2023), and Rebuff (Protectai 2023) enforce safety at the point of LLM output or tool call, catching content-level violations only when the offending path runs. Sandboxed and adversarial evaluations such as ToolEmu (Ruan et al. 2024) and AgentDojo (Debenedetti et al. 2024) measure agent risk by executing behaviors. Agentproof is complementary to both: it analyzes structural properties of the topology before deployment, and its monitor-selection layer (Section 4) decides which runtime monitors are even needed.

Concurrent agent-policy systems. A concurrent line of work enforces policies at the tool-call boundary with formal machinery: Agent-C compiles temporal safety specifications to constraints checked at generation time with conformance guarantees (Kamath et al. 2026); ClawGuard and SecureClaw interpose rule sets and authorization boundaries against prompt injection (Zhao et al. 2026; Ma and Schmid 2026); PolicyGuard trains a compact guardrail model for trajectory-level policy violation detection (Wen et al. 2025); and AgentFlow builds cross-framework agent dependency graphs for static security analysis at scale (Wang et al. 2026). These systems address *how* to enforce or analyze; our study measures *when* the static side of that stack pays off—the base rate of the defects such tools target in public code, and where the error budget actually sits (model extraction). That measurement is orthogonal to, and needed by, every system above.

Model checking and workflow soundness. Classical model checkers—SPIN (Holzmann 1997), NuSMV (Cimatti et al. 2002), CBMC (Kroening and Tautschnig 2014)—verify the same class of properties but require manual translation into Promela/SMV/annotated C; Agentproof extracts the model automatically. Our policy semantics follow LTL on finite traces (De Giacomo and Vardi 2013), with monitoring in

the style of JavaMOP (Jin et al. 2012) and three-valued LTL runtime verification (Bauer, Leucker, and Schallhart 2011). Business-process soundness for Petri nets (van der Aalst 2011) and BPMN (Dijkman, Dumas, and Ouyang 2008) studies analogous reachability properties, but targets visual notations rather than the programmatic API objects agent frameworks expose.

Base rates for static analysis. Industrial experience with conventional static analyzers established that false-positive and actionability rates, not detector cleverness, decide adoption: Coverity's deployment history (Bessey et al. 2010), developer studies of why warnings are ignored (Johnson et al. 2013), and Google's Tricorder lessons (Sadowski et al. 2018). Our study is the agent-workflow instance of that genre—and adds a failure mode those studies did not face: for agent frameworks the *model extraction* step itself dominates the error budget, manufacturing 42% of flags and hiding half the genuine defects.

Agent safety and evaluation. Work on agent safety emphasizes alignment, capability evaluation, and governance (Anthropic 2023; METR 2024; Gabriel et al. 2024). Our contribution is orthogonal and empirical: base-rate measurements of a specific, checkable defect class in public agent workflows, with the estimands and their instrument error separated.

6 Conclusion

Agent frameworks encode behavior as explicit workflow graphs, and AGENTPROOF turns that structure into an analyzable, cross-framework model with structural checks, witness traces, extraction provenance, and a policy DSL with precise LTL_f semantics evaluated both statically—via a conservative graph $\times$ DFA product over finite maximal executions that covers bad prefixes and termination obligations and returns *inconclusive* for behavior it cannot certify—and at runtime. Used as a measurement instrument on 922 workflows from 252 public GitHub repositories, it yields a finding more textured than a simple null: flags are almost never defects (precision 1.1%), yet the instrument also *hides* defects—measured prevalence doubles, to 3.4% of workflows, when checks run on faithful reconstructions—and the genuine defects concentrate in application-like workflows that bind side-effecting tools (7.7%, versus 1.3% among the tutorials and demos that dominate public GitHub). Curated-benchmark defect rates do not stand in for prevalence, and flag counts do not stand in for either.

The diagnosis is consistent across every cut: extraction fidelity, not the check algorithms, is the binding constraint. Edge recall is 0.65 on real code (0.38 on implicit-topology ADK); every structural flag on extracted graphs is an extraction artifact; three of four extractors cannot even express the defects the structural checks target; and the risk-aware gate misses real sensitive paths because tool bindings hide in node bodies. The static machinery keeps a defect-independent job—under an explicit trace-containment premise, the product soundly prunes provably-inert runtime monitors (88% on the curated corpus, all of it alphabet-level today)—positioning pre-deployment analysis as a monitor-selection layer within a static-plus-runtime safety stack.

Limitations. The population is public GitHub, not production systems; two thirds of it is tutorial-grade, which we census and stratify but cannot fully correct. Reference graphs and triage labels are LLM reconstructions with adversarial verification, not human annotations; confidence-stratified sensitivity checks and released per-label justifications mitigate but do not eliminate this, and human annotation of a subsample is the immediate next step. Prevalence CIs are wide at $n{=}119$; the estimates are best read as "rare, but real and concentrated", not as precise rates. We read these results as a redirection—toward higher-fidelity extraction and per-workflow policy instantiation, the direction we pursue in follow-up work on runtime shields.

Artifact. The tool, corpora, mining and validation pipeline, reference-graph reconstructions, triage labels with justifications, and all evaluation scripts will be released under the MIT license.

Ethical Statement

The study mines only *public* GitHub repositories, records exact provenance (repository at commit SHA plus file path), and redistributes only the derived graph abstractions and analysis, never third-party source. No personal data is processed. The four genuine defects we identify are reported without identifying maintainers in the paper; we will disclose them to the affected repositories before publication. The intended impact is defensive—helping developers catch topology-level safety defects before deployment and reason about which runtime safeguards a workflow needs; the honest finding that such defects are rare in public code, yet concentrated in side-effecting application-like workflows, should calibrate, not inflate, claims about static verification's role in agent safety.

References

Anthropic. 2023. Anthropic's Responsible Scaling Policy. https://www.anthropic.com/responsible-scaling-policy. Accessed: 2026-03-01.

Bauer, A.; Leucker, M.; and Schallhart, C. 2011. Runtime Verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology*, 20(4).

Bessey, A.; Block, K.; Chelf, B.; Chou, A.; Fulton, B.; Hallem, S.; Henri-Gros, C.; Kamsky, A.; McPeak, S.; and Engler, D. 2010. A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World. *Communications of the ACM*, 53(2): 66–75.

Cimatti, A.; Clarke, E.; Giunchiglia, E.; et al. 2002. NuSMV 2: An OpenSource Tool for Symbolic Model Checking. In *International Conference on Computer Aided Verification (CAV)*. Springer.

Clarke, E. M.; Grumberg, O.; and Peled, D. A. 1999. *Model Checking*. MIT Press.

CrewAI. 2025. CrewAI documentation. https://docs.crewai.com. Accessed: 2026-02-28.

De Giacomo, G.; and Vardi, M. Y. 2013. Linear Temporal Logic and Linear Dynamic Logic on Finite Traces. In *International Joint Conference on Artificial Intelligence (IJCAI)*.

Debenedetti, E.; Zhang, J.; Balunović, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.

Dijkman, R. M.; Dumas, M.; and Ouyang, C. 2008. Semantics and analysis of business process models in BPMN. *Information and Software Technology*, 50(12): 1281–1294.

Gabriel, I.; et al. 2024. The Ethics of Advanced AI Assistants. *arXiv preprint arXiv:2404.16244*.

Google. 2025. Google Agent Development Kit (ADK) documentation. https://google.github.io/adk-docs/. Accessed: 2026-02-28.

Guardrails AI. 2023. Guardrails AI: Adding Guardrails to Large Language Models. https://github.com/guardrails-ai/guardrails. Accessed: 2026-03-01.

Holzmann, G. J. 1997. The Model Checker SPIN. *IEEE Transactions on Software Engineering*, 23(5): 279–295.

Inan, H.; Upasani, K.; Chi, J.; et al. 2023. LlamaGuard: LLM-based Input-Output Safeguard for Human-AI Conversations. *arXiv preprint arXiv:2312.06674*.

Jin, D.; Meredith, P. O.; Lee, C.; and Roşu, G. 2012. JavaMOP: Efficient Parametric Runtime Monitoring Framework. In *International Conference on Software Engineering (ICSE)*.

Johnson, B.; Song, Y.; Murphy-Hill, E.; and Bowdidge, R. 2013. Why Don't Software Developers Use Static Analysis Tools to Find Bugs? In *International Conference on Software Engineering (ICSE)*.

Kamath, A.; Zhang, S.; Xu, C.; Ugare, S.; Singh, G.; and Misailovic, S. 2026. Enforcing Temporal Constraints for LLM Agents. Preprint. https://github.com/structuredllm/agent-c.

Kroening, D.; and Tautschnig, M. 2014. CBMC – C Bounded Model Checker. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer.

LangChain. 2025. LangGraph documentation. https://docs.langchain.com/oss/python/langgraph/overview. Accessed: 2026-02-28.

Ma, Y.; and Schmid, S. 2026. SecureClaw: Clawing Back Control of LLM Agents. arXiv:2606.09549.

METR. 2024. METR: Model Evaluation and Threat Research. https://metr.org. Accessed: 2026-03-01.

NVIDIA. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications. https://github.com/NVIDIA/NeMo-Guardrails. Accessed: 2026-03-01.

Protectai. 2023. Rebuff: Self-hardening Prompt Injection Detector. https://github.com/protectai/rebuff. Accessed: 2026-03-01.

Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *International Conference on Learning Representations (ICLR)*.

Sadowski, C.; Aftandilian, E.; Eagle, A.; Miller-Cushon, L.; and Jaspan, C. 2018. Lessons from Building Static Analysis Tools at Google. *Communications of the ACM*, 61(4): 58–66.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.

van der Aalst, W. M. P. 2011. *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Springer.

Wang, S.; Hou, X.; Zhao, Y.; Cheng, X.; and Wang, H. 2026. AgentFlow: Building Agent Dependency Graphs for Static Analysis of Agent Programs. arXiv:2607.01640.

Wen, X.; Mo, W. J.; Xie, Y.; Qi, P.; and Chen, M. 2025. Towards Policy-Compliant Agents: Learning Efficient Guardrails for Policy Violation Detection. arXiv:2510.03485.

Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; et al. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *Conference on Language Modeling (COLM)*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

Zhao, W.; Li, Z.; Zhang, P.; and Sun, J. 2026. ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents Against Indirect Prompt Injection. arXiv:2604.11790.

Reproducibility Checklist

Instructions for Authors:

This document outlines key aspects for assessing reproducibility. Please provide your input by editing this `.tex` file directly.

For each question (that applies), replace the "Type your response here" text with your answer.

Example: If a question appears as

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
Type your response here
```

you would change it to:

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
yes
```

Please make sure to:

- Replace ONLY the "Type your response here" text and nothing else.
- Use one of the options listed for that question (e.g., **yes**, **no**, **partial**, or **NA**).
- **Not** modify any other part of the `\question` command or any other lines in this document.

You can `\input` this .tex file right before `\end{document}` of your main file or compile it as a stand-alone document. Check the instructions on your conference's website to see if you will be asked to provide this checklist with your paper or separately.

---

1. General Paper Structure

1.1. Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA) yes

1.2. Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes/no) yes

1.3. Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no) yes

2. Theoretical Contributions

2.1. Does this paper make theoretical contributions? (yes/no) no

If yes, please address the following points:

2.2. All assumptions and restrictions are stated clearly and formally (yes/partial/no) NA

2.3. All novel claims are stated formally (e.g., in theorem statements) (yes/partial/no) NA

2.4. Proofs of all novel claims are included (yes/partial/no) NA

2.5. Proof sketches or intuitions are given for complex and/or novel results (yes/partial/no) NA

2.6. Appropriate citations to theoretical tools used are given (yes/partial/no) NA

2.7. All theoretical claims are demonstrated empirically to hold (yes/partial/no/NA) NA

2.8. All experimental code used to eliminate or disprove claims is included (yes/no/NA) NA

3. Dataset Usage

3.1. Does this paper rely on one or more datasets? (yes/no) yes

If yes, please address the following points:

3.2. A motivation is given for why the experiments are conducted on the selected datasets (yes/partial/no/NA) yes

3.3. All novel datasets introduced in this paper are included in a data appendix (yes/partial/no/NA) partial

3.4. All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no/NA) yes

3.5. All datasets drawn from the existing literature (potentially including authors' own previously published work) are accompanied by appropriate citations (yes/no/NA) NA

3.6. All datasets drawn from the existing literature (potentially including authors' own previously published work) are publicly available (yes/partial/no/NA) NA

3.7. All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisficing (yes/partial/no/NA) NA

4. Computational Experiments

4.1. Does this paper include computational experiments? (yes/no) yes

If yes, please address the following points:

4.2. This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting (yes/partial/no/NA) NA

4.3. Any code required for pre-processing data is included in the appendix (yes/partial/no) yes

4.4. All source code required for conducting and analyzing the experiments is included in a code appendix (yes/partial/no) yes

4.5. All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no) yes

4.6. All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from (yes/partial/no) partial

4.7. If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results (yes/partial/no/NA) yes

4.8. This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks (yes/partial/no) partial

4.9. This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics (yes/partial/no) yes

4.10. This paper states the number of algorithm runs used to compute each reported result (yes/no) yes

4.11. Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information (yes/no) yes

4.12. The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank) (yes/partial/no) NA

4.13. This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments (yes/partial/no/NA) yes